CODE WIKI · VERL-PROJECT/VERL-OMNI

VeRL-Omni：多模态生成式强化学习源码导读

面向准备贡献或定制 VeRL-Omni 的工程师：从一条 Diffusion V1 FlowGRPO 训练迭代出发，解释 Hydra、Ray、TransferQueue、rollout、奖励、训练 worker 与模型适配器怎样协作。

verl-project/verl-omni @ d65b201

怎么读这份导读

这份导读写给第一次进入 VeRL-Omni、却需要改训练配方、接入一个新模型或定位吞吐 / 显存问题的工程师。它不按目录逐个罗列；主线是一条 **Diffusion V1 + FlowGRPO** 的在线训练迭代，因为这是当前默认路径。读完后，你应能判断某项改动属于配置、pipeline adapter、rollout 服务、reward，还是分布式训练核心。

要点 · 固定版本与边界

- 源码固定为 `verl-project/verl-omni` @ `d65b2017fe09fc30a60f8aea70b445b604b5db45`（2026-11-09）。所有源码链接固定到这个提交。
- 主干：`main_diffusion_v1 → Ray task runner → TransferQueue → agent loop / vLLM-Omni rollout → reward → actor update`。入口见 `run_diffusion_v1`。
- VeRL-Omni 不是一个自给自足的训练运行时：它固定依赖 `verl`、`vllm` 和 `vllm-omni`，版本证据见 `pyproject.toml 依赖`。这些上游仓库的内部实现不在本报告的可审计范围内。

推荐顺序：先读“全景”和“端到端生命周期”，再按你的任务选“配置”“rollout”“奖励”“worker / adapter”。最后两章说明贡献边界与许可证。

全景：问题、目标、限制

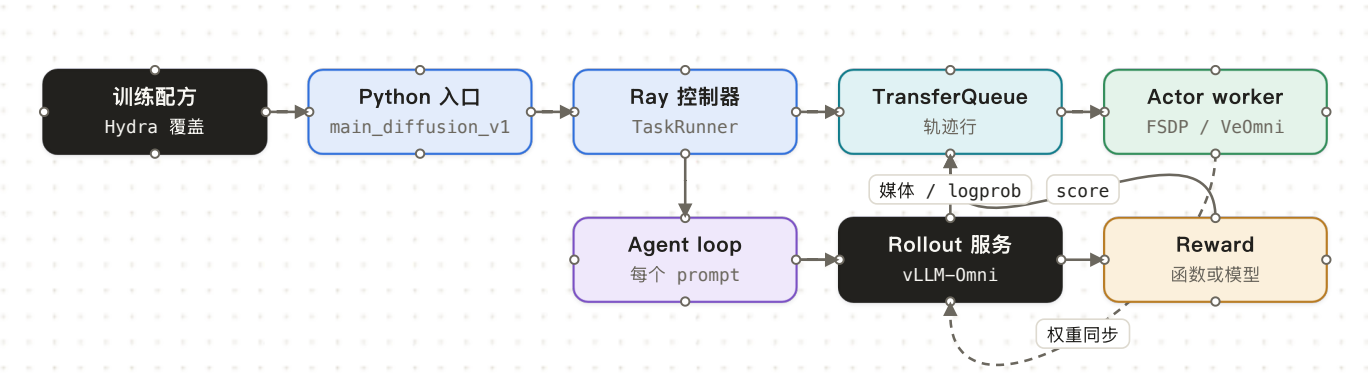
没有它时的问题

生成式多模态 RL 把三种本来就昂贵的工作绑在了一起：采样媒体、打分媒体、更新大模型。图像 / 视频的 rollout 不像文本 token 流那样轻；同一张 GPU 上同时常驻 actor、推理引擎和 reward model 又容易 OOM。VeRL-Omni 的回答不是单一算法，而是一套可组合的训练运行骨架。

环节	现实约束	VeRL-Omni 的处理
rollout	图像、视频、音频的生成延迟高，`n` 个组内样本必须保留对应关系	vLLM-Omni rollout + TransferQueue 行记录，按 prompt group 完成采样
reward	评分器可能是 Python 函数、HTTP 服务或 GPU 模型，延迟和资源形态不同	`OmniRewardLoopManager` 统一调度命名 / 原生 / engine reward
actor update	FSDP/VeOmni、LoRA、reference policy、不同 diffusion loss 的输入契约不同	Ray worker group 只暴露 actor 推理和更新接口，adapter / loss registry 吸收差异
分布式资源	rollout 和训练可共享 GPU，也可独占 GPU 并异步运行	`sync` 与 `separate_async` 两套 V1 生命周期；后者通过 checkpoint engine 同步权重

README 把项目定位为“面向 diffusion 与 omni-modality 的 RL 训练框架”，并明确建立在 `verl` 之上。README 定位 它支持的模型 / 算法矩阵不是一个核心引擎的功能开关，而是许多 pipeline adapter 与 recipe 的组合。README 支持矩阵

VeRL-Omni 的公开系统边界



深色节点是此仓库调用、但不在此仓库实现的边界；入口强制初始化 TransferQueue 的证据见 `main_diffusion_v1.py`，rollout registry 见 `replica.py`。

要求、目标、限制矩阵

维度	Diffusion V1	Omni / AR 路径	影响贡献者的结论
核心目标	媒体生成的 online RL；默认 V1	文本、图像、音频、视频的统一模型训练	不要把 diffusion 假设硬编码进共享 trainer
主要数据	`DataProto` + TransferQueue 的轨迹行	另一套 omni trainer / dataset 适配	先选 `main_diffusion_v1` 还是 `main_omni`
默认并行	Ray + FSDP2；rollout 可用 TP / USP	可选 VeOmni / Megatron 路径	配置组合需要启动前校验
显存约束	sync 模式在采样后让 rollout 休眠 / 卸载	依模型和 engine 而变	生命周期 hook 不能随意挪动
可扩展点	model adapter、loss、advantage、reward、agent loop	dataset transform、omni pipeline、worker	优先 registry / `external_lib`，不要 fork trainer
当前许可证	Apache-2.0	Apache-2.0	模型权重、外部服务另有条款

类比 · 把一次训练看成“共享冷库 + 两个车间”

`TransferQueue` 像冷库：它保存带身份标签的 rollout 成品，训练车间按组取货，质检车间可以把分数补回同一箱货。rollout 服务是生产车间，actor worker 是加工车间；在 `sync` 模式，两车间轮流占用同一块厂房，所以采样结束后必须让 rollout 休眠。这个映射对应 `TransferQueue` 行、`ReplayBuffer` 和 `checkpoint_manager.sleep_replicas()`，而不是一个普通 Python 队列。 `sync 生命周期`

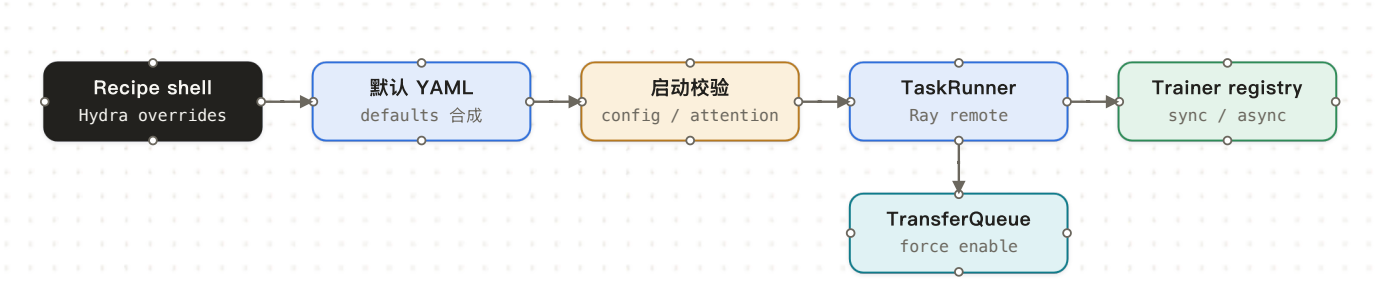
入口与配置：先决定要启动哪一种训练

职责与难点

入口的职责是把 Hydra 配置解析成一个可运行、可验证的 Ray 作业；难点是“同样一个 YAML”在不同设备、profiling、trainer mode 和模型后端下并不都有效。`main()` 会自动识别设备、resolve Hydra、执行通用配置和 attention 一致性验证，然后按 `trainer.use_v1` 选择 V1 或 deprecated 的 V0。`main`

V1 runner 在 `ray.init()` 前强制设置 `config.transfer_queue.enable = True`，把 `TRANSFER_QUEUE_ENABLE=1` 放进 Ray runtime environment，再 remote 一个 `DiffusionTaskRunnerV1`。`runner 初始化` 这解释了一个容易踩坑的事实：YAML 默认值仍是 `false`，但 V1 会覆盖它。`TransferQueue 默认配置`

配置从 shell 到 trainer



示例脚本明确调用 V1 入口并设置 `trainer.v1.trainer_mode=sync`，见 `Qwen-Image V1 recipe`；trainer mode 默认值与 async 限制见 `V1 配置`。

关键契约

名称	作用	证据
`trainer.use_v1`	默认 `true`；设为 `false` 走 legacy V0	`config`
`trainer.v1.trainer_mode`	已注册的 `sync` 或 `separate_async`	`trainer registry`
`algorithm.trainer_type`	`policy_gradient` 或 `direct_preference`	`algorithm 配置`
`actor_rollout_ref`	actor、rollout、ref、model 的组合配置根	`actor_rollout_ref`
`reward`	custom function、named models、资源池选择	`reward 配置`

容易误解： V1 的 `sync` 不是“没有 Ray”；它仍使用 Ray 和异步 rollout server，只是不允许 partial rollout，并在一个更新周期内等完整组。V1 文档描述了 sync / separate-async 的差异和 `parameter_sync_step` 的批大小约束。`架构说明` `async 约束`

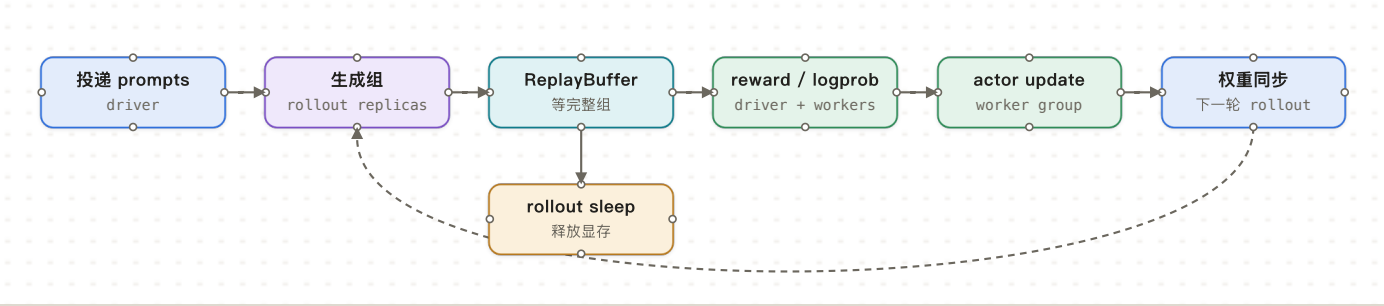
V1 trainer：把组完整性和训练生命周期放在 driver

职责、难点与原理

`PolicyGradientDiffusionTrainerV1` 是 V1 的共享 driver。它初始化 replay buffer、计算是否要 reference/reward/teacher policy，并以 lifecycle hook 把“什么时候采样、什么时候 sleep、什么时候同步权重”留给 mode 子类。`V1 base 初始化`

`sync` 子类很小，正是设计意图：初始化和每一步结束时同步权重；采样结束后 sleep rollout replicas。`sync hooks` `separate_async` 则要求独立 rollout GPU 和非 naive checkpoint backend。`separate_async 配置约束`

V1 sync 一步的资源时序



`fit()` 的外层 step、存储清理与日志见 `fit`；每步先 feed、再 sample / train 的代码在 `step`。

契约与连接

抽象	输入 / 输出	为什么重要
`ReplayBuffer`	按状态轮询 TransferQueue，返回 `KVBatchMeta`	它保证训练消费的是完成的 rollout 组；sync 默认轮询 0.05 秒
`KVBatchMeta`	partition、keys、tags	是队列行的轻量索引，真正张量仍在队列中
`DataProto`	从队列 materialize 的训练 batch	后续 old logprob、ref、advantage、actor 都使用它
`parameter_sync_step`	一个外层 step 里的 local updates	async 模式按周期保持 / 刷新采样策略

`_build_replay_buffer()` 对 sync 使用 `ReplayBuffer`，对 separate async 使用 `ReplayBufferAsync`；若未配置轮询时间，sync 选择 0.05 秒、async 选择 2 秒。`_build_replay_buffer` `step()` 又强制 `train_batch_size` 能被 `parameter_sync_step` 整除。`step 批大小规则`

注意 · 容易误解：不能“顺手”重排 rollout 行

FlowGRPO 的 advantage 依赖同一 prompt 的多个 rollout；direct preference 还依赖 chosen/rejected 成对进入 micro-batch。代码因此会 pad 以适配 DP / mini-batch，再把补行 advantage 置零，并在 whole-group loss 下关闭 shuffle。`balance 与 padding` `actor update 分组规则`

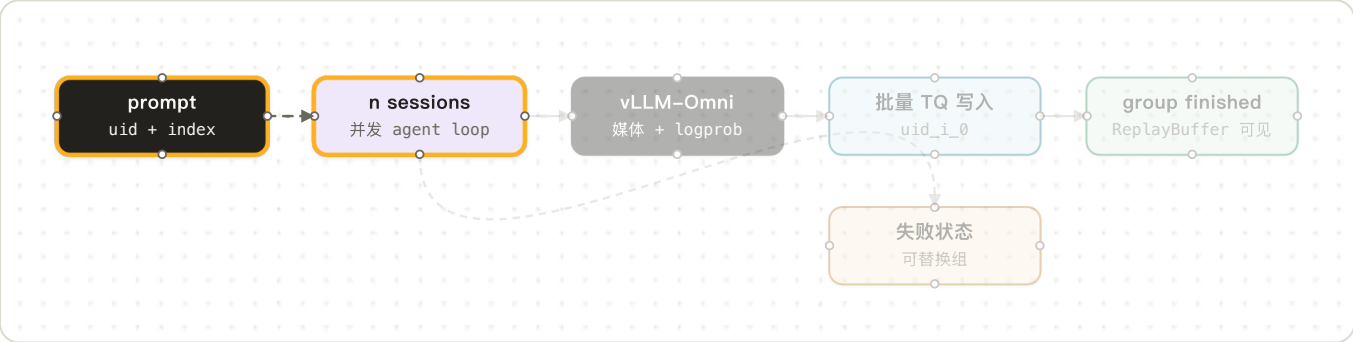
Rollout 与 TransferQueue：轨迹如何离开推理服务又回到训练

职责、难点与原理

`DiffusionAgentLoopWorkerTQ` 把一个 prompt 展开为 `rollout.n` 个 session；每一个完成的 diffusion 输出被组装成队列行。它用 `uid_session_output` 作为 row key，保存 prompt ids、生成媒体、可选 rollout logprob、reward score、媒体 metadata 和生成时的 policy step。`_run_prompt` `_write_trajectories_to_tq`

难点不是“把 tensor 放进队列”，而是保持 group 原子性、支持失败、支持音视频等不同 artifact，同时避免每个 session 一次 RPC。实现会先并发 session，再按字段签名批量写入；成功 / 失败会回写 prompt 状态。`组状态`

▶ 一个 prompt 的 `n` 次生成如何成为可训练样本 1 / 4



1. 分配确定性种子

训练 batch 会附加 step-local 的 base seed 和 prompt index。worker 用 `prompt_index * n + session_id` 派生每次采样的 seed，避免不同 worker chunk 重用随机噪声。`生成参数和 seed` `每个 session 的 seed`

2. 并发生成

每个 prompt 生成 `n` 个 session，`asyncio.gather()` 汇总成功结果和异常；rollout engine 的 registry 以 `vllm_omni` 名称加载具体 replica。`并发组` `rollout manager`

3. 一次写入一组

同字段的 session 被放入一次 `async_kv_batch_put`。每行携带 `min_global_steps/max_global_steps`，供后续 off-policy 统计。`队列行格式`

4. 完整组可消费

driver 用 `KVBatchMeta` materialize 成 `DataProto`；训练完把 score、advantage、return 回写，日志 / dump 后清理这些 keys。`消费和回写` `清理时机`

关键数据契约

字段	谁写入	谁消费
`responses`、`prompts`	agent loop	adapter / actor
`rollout_log_probs`	rollout（若需要）	rollout correction 或 loss
`rm_scores` / reward 信息	streaming reward 或后续 colocated reward	advantage、metrics
`uid`、`extra_fields`、policy steps	agent loop	分组、可观测性、媒体 dump
`advantages`、`returns`	trainer	actor update、最终 dump

从 `KVBatchMeta` 还原的函数明确说明：张量 batch 可包含 `responses`、`rollout_log_probs`、`rm_scores`、embeds 等，非张量部分含 uid、reward model、data source、extra fields。`diffusion_tq_batch_to_dataproto` 写回函数只写指定 fields，避免将不应持久化的 driver 状态塞回队列。`put_dataproto_fields_to_tq`

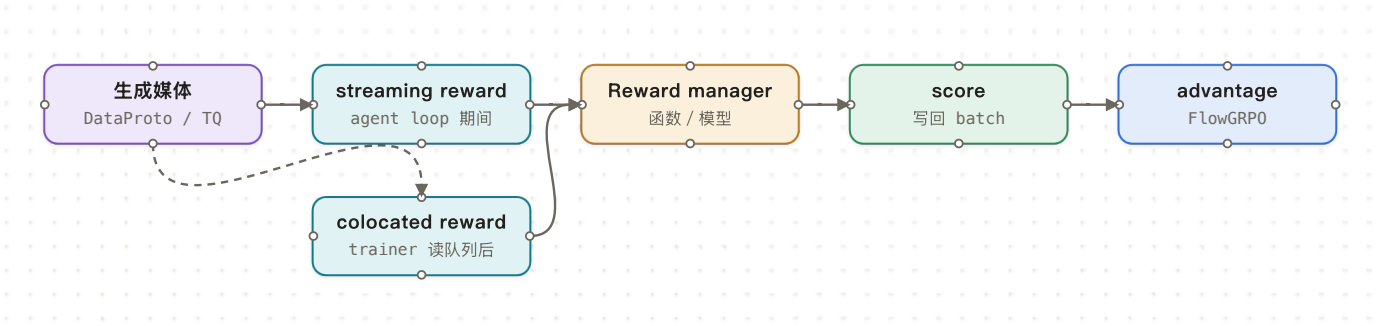
Reward：同一套接口容纳本地函数、HTTP 与模型

职责、难点与原理

奖励层把“生成媒体变成可学习的标量或组件”从 rollout 和 actor 中拆出去。`OmniRewardLoopManager` 支持 legacy reward、命名 reward model、engine backend、native backend 和 accelerator workers；它还检查某些聚合方式只可用于 OmniNFT。`OmniRewardLoopManager 初始化`

多 reward 的难点是资源并不均匀：某些是 CPU 函数、某些是共享 engine、某些是独占 native GPU 模型。manager 会按 model placement 分 worker group，区分 shared engine terms 和 native terms。`reward worker 分组`

Reward 的两种接入时机



streaming handle 只在配置允许时暴露，见 `reward_loop_worker_handles`；colocated 路径在 trainer 内被调用，见 `_train_sampled_batch 的 reward 阶段`。

契约与坑

配置	含义	证据
`custom_reward_function`	指向一个 Python 文件与函数名	`custom reward`
`reward_functions`	多项 reward term 的声明	`multi reward`
`models`	命名模型与 backend / placement	`named models`
`aggregation`	`weighted_sum` 或保留组件	`aggregation`

容易误解： 只有“有 reward model”不代表一定在 rollout 时流式评分。`reward_loop_worker_handles` 会根据 `streaming_reward_enabled` 返回 `None`；此时 V1 在读取队列后调用 colocated scorer。`streaming 条件` 如果把 reward worker 与 actor / rollout 混在同一资源池，sync 模式的 sleep / wake 顺序也成为显存正确性的一部分。`显存生命周期说明`

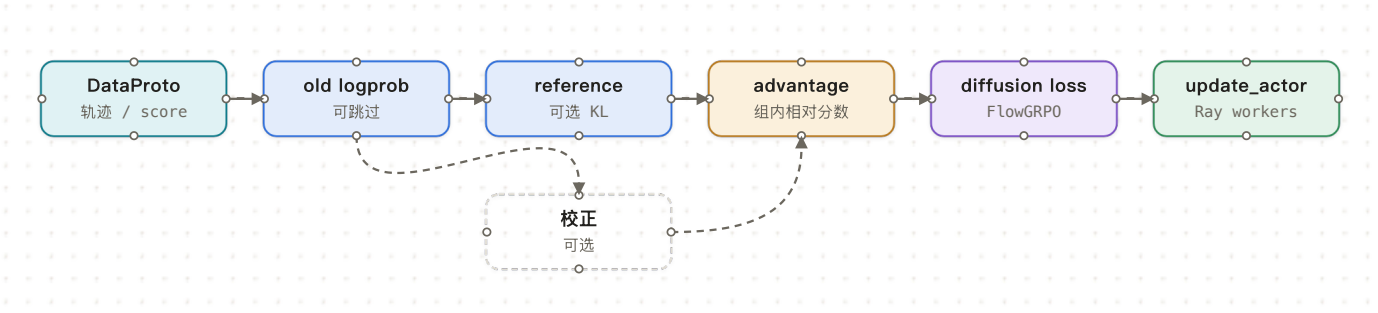
优势、策略与 actor worker：从分数变成梯度

职责、难点与原理

对于 FlowGRPO，driver 先将 sample score 扩到每个 diffusion timestep，再按同 uid 的 rollout group 计算 group-relative advantage。`_compute_advantage` 默认实现可以选择 global 或 group std 来归一化。`FlowGRPO advantage`

之后 trainer 依 recipe 执行 old logprob、rollout correction、reference logprob、teacher scoring，最后调用 actor worker 的 `update_actor`。整个顺序就在 `_train_sampled_batch()`；它是理解该项目的最高价值函数。`训练 batch 主路径`

FlowGRPO 的计算链



old logprob 通过 actor 的 inference 接口重算，见 `_compute_old_log_prob`；reference 可复用去 LoRA 的 actor 或单独 worker，见 `_compute_ref_log_prob`。

契约

抽象	必需输入	产出 / 约束
`DiffusionLossFn`	worker model output + batch fields	各 loss 声明缺失字段时 fail fast
`FlowGRPOLoss`	`log_probs`、`old_log_probs`、`advantages`	clipped policy objective
`DiffusionAdvantageEstimator`	`sample_level_rewards`、group index	每 timestep 的 advantage / return
actor worker	`TensorDict` 和 micro-batch 元数据	metrics，经 driver 聚合

loss registry 与输入校验见 `DiffusionLossFn 与 registry`；`FlowGRPOLoss` 明确要求 `log_probs`、`old_log_probs` 与 `advantages`。`FlowGRPOLoss` actor 更新会把 `ppo_mini_batch_size` 乘上 rollout `n`，并把 actor metrics 改名前缀。`_update_actor`

类比 · 一个 rollout group 像同一道考题的多份答卷

奖励的绝对值不直接说明“这张图该增大还是减小概率”；FlowGRPO 把同一 prompt 的多个 session 当作同一道题的多份答卷，用组均值做基线。 `uid` 是题号， `n` 是答卷数，归一化后的差值是学习信号。映射可在 advantage 实现的 `id2score` 分组逻辑直接看到。 `group-relative advantage`

容易误解： direct preference 不走这条 old logprob 路径。代码在 `trainer_type=direct_preference` 时改用 `_train_direct_preference_batch()`，DPO 等 loss 需要 `noise_pred`；offline DPO 仍刻意留在 V0。 `direct preference 边界` `direct preference 路径`

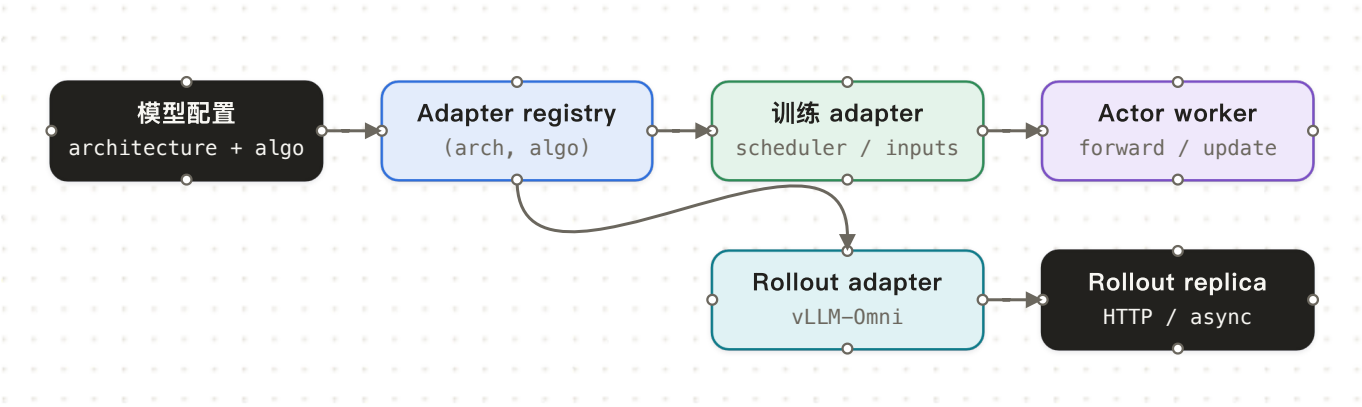
模型 adapter 与 rollout backend：可扩展性放在边缘，而不在 trainer 分叉

职责、难点与原理

Diffusion 模型的 scheduler、text condition、latent 格式和一次 forward 的语义差别很大。`DiffusionModelBase` 因此用 `(architecture, algorithm)` 注册 adapter：architecture 通常来自模型的 `model_index.json`，algorithm 来自配置。`adapter registry` 在初始化 tokenizer / processor 前，trainer 也先解析 architecture 并调用 adapter 的 `prepare_processor_files()`。`模型与 processor 初始化`

adapter 必须实现 scheduler、timestep、输入准备、forward+采样上一步等抽象方法；这使 FlowGRPO trainer 不需要知道 Qwen-Image、SD3、Wan 或视频模型的内部输入格式。`DiffusionModelBase 抽象契约`

新 diffusion 模型的接入边界



不匹配的 adapter 会在 registry 查找时抛出包含已注册 keys 的错误，见 `get_class_by_name`；rollout output 的媒体 artifact 合同见 `DiffusionOutput`。

怎样定制

需求	应修改的边界	不该修改的边界
支持新 diffusion architecture	`DiffusionModelBase` 子类 + 对应 training / rollout adapter	V1 trainer 的循环
新 policy-gradient 算法	advantage estimator、loss、配置、测试	复制一份 Ray trainer
新 reward	reward function / reward manager / named model config	把评分代码塞进 pipeline
新 agent 工作流	agent loop manager 或 custom class FQN	破坏 TQ row schema
新后端	worker engine / rollout replica registry	假设 `vllm_omni` 是唯一实现

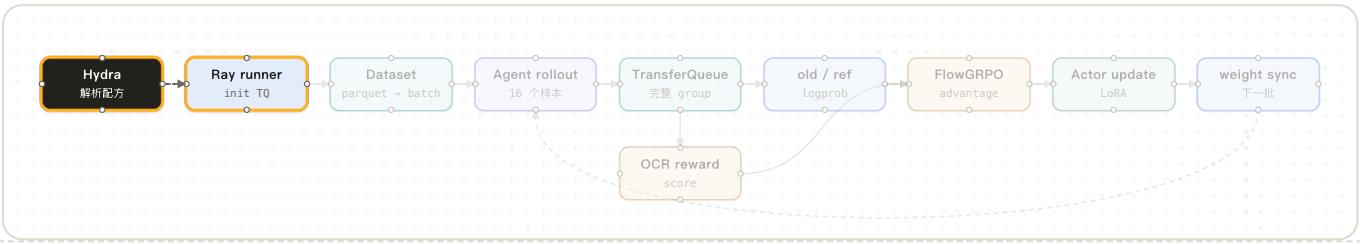
仓库提供的贡献文档把“新 diffusion model”“新 policy-gradient algorithm”“新 direct-preference algorithm”“新 omni model”拆成不同路线。集成指南索引 外部 adapter 可以通过 `external_lib` 动态导入，而不是改仓库 registry。

`external_lib`

端到端生命周期：Qwen-Image V1 FlowGRPO 的一次迭代

以下以仓库中的 Qwen-Image OCR LoRA V1 recipe 为例；它使用 4 GPU、rollout `n=16`、vLLM-Omni rollout、OCR reward。 完整配方 这是从源码读出的控制流，不代表每个模型 / 算法配方都使用同一组参数。

▶ 一条训练迭代的完整路径 1 / 6



1. 1：解析与建图

脚本从 `python3 -m verl_omni.trainer.main_diffusion_v1` 启动。入口 resolve 配置、初始化 Ray，并在 TaskRunner 中初始化 TransferQueue、trainer 和 agent loop manager。 `TaskRunner.run`

2. 2：读取 prompt batch

trainer 用 tokenizer / processor 建 dataset，并用 `StatefulDataLoader` 取 generation batch；训练步数可来自 epochs 或显式 `total_training_steps`。 `_init_dataloader`

3. 3：生成并写入队列

driver 先 `_add_batch_to_generate()`；agent loop 对每个 prompt 启动 session，生成 tensor 和 metadata 后批量写入 TransferQueue。 `feed` `generate_sequences`

4. 4：评分与取样

ReplayBuffer 等完整 group；如果是 colocated reward，trainer 在 materialize 后计算 score。否则 agent loop 的 streaming scorer 已把结果带入队列行。 `sample + reward`

5. 5：计算训练张量

trainer 依配置重算 old logprob、可选 reference logprob，扩展 score 到 timestep，并计算 FlowGRPO advantage。 `logprob 到 actor update`

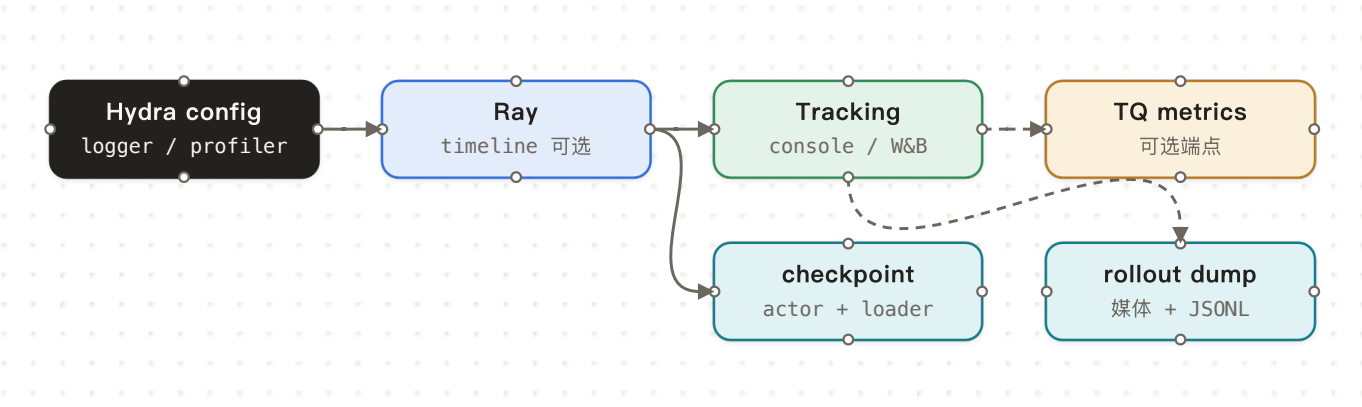
6. 6：更新并同步

actor worker 更新 LoRA / 参数；sync mode 的 `on_step_end()` 再唤醒 rollout replica 并同步新权重。该顺序是避免 actor update 与大 rollout 权重同驻留而 OOM 的内存协议。 `同步 hook`

部署、运行与观测边界

项目的正常环境是 Python 3.11+、CUDA 13+ 的 GPU 训练环境；官方安装路径使用 `uv`，首先安装固定 vLLM，再安装项目 GPU extra。 系统要求 安装命令 `pyproject.toml` 同时固定了 `vllm==0.30.0`、特定提交的 `vllm-omni` 和 `verl`。 依赖 pins

运行时的可观测性与持久化



默认 logger 为 console + W&B，checkpoint 和 rollout dump 配置见 `trainer 配置`；TQ metrics/backend 配置见 `TransferQueue backend`。

V1 的 `fit()` 在更新后可 checkpoint、验证、导出 rollout 数据，随后清除本步 keys。 `fit 的持久化和清理` 不应把队列当作永久数据湖：默认 SimpleStorage 是进程内 / ZMQ 方式；另一个 MooncakeStore 仍标为 experimental。 队列存储 backend

贡献与定制

真实开发路径

官方贡献流程建议 GPU 环境下安装 `vllm==0.30.0`、editable 包和 `pre-commit`；贡献前运行相关测试和 pre-commit。开发环境 lint 与测试 文档构建命令也在贡献指南中。 文档构建

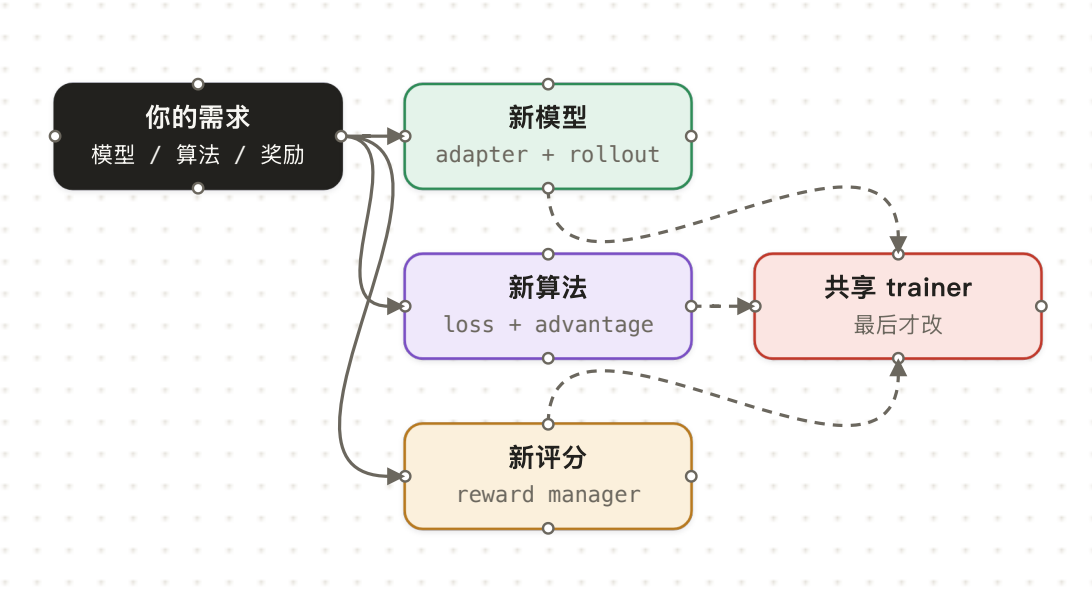
目标	起点	首先验证
新模型	integration guide + adapter registry	model / rollout 输入、scheduler、最小 recipe
新算法	loss / advantage registry	输入字段、组完整性、CPU 单测 + GPU smoke
新评分器	reward config / manager	placement、媒体类型、failover、聚合含义
性能优化	先 profile，再动 lifecycle	是否破坏 sleep / wake 与 weight sync
新异步拓扑	separate_async + checkpoint engine	policy staleness、完整组、权重同步后端

注意 · 共享训练核心有明确的贡献边界

上游 `AGENTS.md` 要求共享的 `trainer/`、`workers/`、`agent_loop/` 核心保持 model- 和 algorithm-agnostic，优先 registry 而不是 fork；也要求检查重复工作、人工审阅变更，且未经明确授权不得修改 CI / agent instructions。 贡献政策与 protected surfaces

定制路线

选择最小的改动面



这不是说永远不能改 core，而是先利用现有 extension contracts。模型 adapter 的注册点见 `registry`，loss / advantage 注册点见 `loss 与 advantage registries`。

开源许可证与第三方边界

当前仓库根目录是 Apache License 2.0。`LICENSE` 项目元数据也声明 `Apache-2.0`。`project metadata` GitHub API 的 `LICENSE` 路径历史显示，2026-04-29 的提交 `122caa19923b3eb2bfff0789b69a9c73b05df220` 将 Apache-2.0 文本加入此仓库；本报告未找到、也不声称有更早的本仓库许可证历史。

场景	Apache-2.0 下通常可以做什么	需要保留 / 注意
自用、实验	可以使用和修改	风险自担；第三方依赖 / 模型仍适用自己的条款
给上游提 PR	可以	项目贡献规则、人工审阅、测试与署名要求
公开 fork	可以	附许可证、保留 copyright/patent/trademark notices，标明修改
发布安装包	可以	附许可证和相关 NOTICE（若分发中存在）
对外提供训练服务	Apache-2.0 不像 AGPL 那样要求公布服务端源码	仍要审查模型权重、数据、vLLM / verl 等第三方许可
使用项目名 / Logo	只限合理描述来源	Apache-2.0 不授予商标许可

上表是 Apache-2.0 的工程摘要，不是法律意见。许可证文本规定分发要求、贡献默认条款和商标例外。`Apache 分发条款` `贡献与商标` 仓库还固定了 Git 依赖和模型生态软件；这些都不是根 LICENSE 自动覆盖的对象。`第三方依赖边界`

附录：源码索引

主题	从这里开始
V1 入口 / Ray	`main_diffusion_v1.py`
V1 trainer 主循环	`trainer_base.py`
sync 生命周期	`trainer_sync.py`
separate async 生命周期	`trainer_separate_async.py`
队列数据转换	`tq_utils.py`
diffusion agent loop	`diffusion_agent_loop_tq.py`
reward 编排	`reward_loop.py`
loss / advantage	`diffusion_algos.py`
模型 adapter 协议	`model_base.py`
rollout replica	`replica.py`
默认训练配置	`diffusion_trainer.yaml`
默认 rollout 配置	`diffusion_rollout.yaml`
安装与 V1 文档	`install.md`、`diffusion_v1.md`
贡献边界	`AGENTS.md`、`CONTRIBUTING.md`